Package 'DegradSim'

November 12, 2026

Title Simulation of Levy and Time-Scaled Levy Degradation Processes

Version 0.1.0

Description Exact simulation, at given inspection times, of one or several degradation trajectories of Levy processes (gamma, Wiener, inverse Gaussian, variance-gamma, Meixner, normal inverse Gaussian and compound Poisson processes) and of time-scaled Levy processes X_t = Y_{A(t)}, obtained with a time-scaling function A from a gamma, Wiener or inverse Gaussian process (Kahle, Mercier and Paroissin, 2016, ISBN:978-1-84821-888-8; Cha and Mercier, 2019, <doi:10.1017/apr.2019.21>); inspection times with a chosen pattern, plotting of the trajectories and extraction of the simulated data.

License GPL-3

URL https://github.com/rytracetito/levy_packages/tree/main/DegradSim, https://github.com/rytracetito/levy_packages

BugReports https://github.com/rytracetito/levy_packages/issues

Encoding UTF-8

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.3

Imports ggplot2, levyBridge, Runuran, stats, utils

Suggests testthat (>= 3.0.0)

Remotes github::rytracetito/levy_packages/levyfit, github::rytracetito/levy_packages/levyBridge

Config/testthat/edition 3

Author Amedée Rytrace Tito [aut, cre], Christian Paroissin [aut]

Maintainer Amedée Rytrace Tito <rytracetito5@gmail.com>

Contents

DegradSim-package 2
inspection_times 8
plot_plan 10
plot_traj 11

sim_cpoisson 12
sim_gamma 15
sim_ig 19
sim_meixner 23
sim_nig 26
sim_vg 29
sim_wiener 33
traj_df 37

Index **39**

`DegradSim-package` *DegradSim: simulation of Levy and time-scaled Levy degradation processes*

Description

`DegradSim` simulates exactly, at given inspection times, one or several degradation trajectories of two families of processes:

1. **Levy processes**: gamma, Wiener, inverse Gaussian, variance-gamma, Meixner, normal inverse Gaussian and compound Poisson processes;
2. **time-scaled Levy processes** $X_t = Y_{A(t)}$, where Y is a Levy process and A a time-scaling function: time-scaled gamma, Wiener and inverse Gaussian processes.

Details

The inspection times can be irregular and differ from one trajectory to another; `inspection_times()` builds them with a chosen pattern. The simulated trajectories can be printed, plotted and extracted as a data frame.

Levy processes

Definition (Sato, 2001, p. 3). A real stochastic process $(X_t)_{t\geq 0}$ is a Levy process if

1. it has independent increments: for all $n \geq 1$ and $0 \leq t_0 < t_1 < \cdots < t_n$, the random variables $X_{t_0}, X_{t_1} - X_{t_0}, \ldots, X_{t_n} - X_{t_{n-1}}$ are independent;
2. it starts at the origin: $X_0 = 0$ almost surely;
3. it is time homogeneous: the distribution of $(X_{s+t} - X_s)_{t\geq 0}$ does not depend on s;
4. it is stochastically continuous: for all $\varepsilon > 0$, $P(|X_{s+t} - X_s| > \varepsilon) \to 0$ as $t \to 0$;
5. its trajectories $t \mapsto X_t$ are right-continuous with left limits (càdlàg), almost surely.

Infinitely divisible distribution (Sato, 2001, p. 4). A distribution μ on $\mathbb{R}$ is infinitely divisible if, for every integer $n \geq 1$, there is a distribution μ_n such that $\mu = \mu_n^{n*}$, the n-fold convolution of μ_n: a random variable with distribution μ has the distribution of the sum of n independent random variables with distribution μ_n. For a Levy process, the distribution of X_t is infinitely divisible for every t; conversely, every infinitely divisible distribution is the distribution of X_1 for a Levy process, unique in distribution (Sato, 2001, Theorems 1.1 and 1.2).

The simulation functions, with the distribution of an increment over a time $\Delta t = t - s$:

Function	Process	Parameters	$X_t - X_s$
`sim_gamma()`	gamma	a, b	$\Gamma(a\Delta t, b)$

`sim_wiener()`	Wiener	`mu`, `sigma`	$\mathcal{N}(\mu\Delta t, \sigma^2\Delta t)$
`sim_ig()`	inverse Gaussian	`mu`, `eta`	$IG(\mu\Delta t, \eta\Delta t^2)$
`sim_vg()`	variance-gamma	`theta`, `sigma`, `nu`	$\theta G_{\Delta t} + \sigma B_{G_{\Delta t}}$
`sim_meixner()`	Meixner	`a`, `b`, `d`, `m`	$Meixner(a, b, d\Delta t, m\Delta t)$
`sim_nig()`	NIG	`alpha`, `beta`, `delta`, `mu`	$NIG(\alpha, \beta, \delta\Delta t, \mu\Delta t)$
`sim_cpoisson()`	compound Poisson	`lambda`, `rjump`	sum of $Poisson(\lambda\Delta t)$ jumps

For the variance-gamma process, B is a standard Brownian motion and G an independent gamma process with $G_{\Delta t} \sim \Gamma(\Delta t/\nu, 1/\nu)$ (see `sim_vg()`).

Other Levy processes

There are other Levy processes. `DegradSim` does not simulate them, except the Poisson process, given by `sim_cpoisson()` with jumps equal to 1:

- the **Poisson process**, increments $Poisson(\lambda\Delta t)$ (Hoyle, 2010, section 2.11);
- the **Cauchy process**, increments $Cauchy(0, \gamma\Delta t)$, of whose mean is not defined and whose second moment is infinite (Hoyle, 2010, section 2.9);
- the **α-stable processes**, of infinite variance for $\alpha < 2$; for $\alpha < 1$, the stable subordinator is increasing (Samorodnitsky and Taqqu, 1994; exact simulation by Chambers, Mallows and Stuck, 1976);
- the **generalized hyperbolic (GH) processes**, whose distribution at time 1 is generalized hyperbolic (Barndorff-Nielsen, 1977); the NIG and the variance-gamma processes are special or limiting cases;
- the **tempered stable processes**, stable processes whose large jumps are tempered by an exponential factor (Koponen, 1995; Rosiński, 2007); the gamma and inverse Gaussian processes are tempered stable subordinators;
- the **Levy jump-diffusion processes**, a Brownian motion with drift plus a compound Poisson process, with normal jumps (Merton, 1976) or double exponential jumps (Kou, 2002);
- the **CGMY process** (Carr, Geman, Madan and Yor, 2002), a tempered stable process that contains the variance-gamma process as a special case.

Time-scaled Levy processes

Time-scaling function. Let $A : [0, +\infty[\to [0, +\infty[$ be a non-decreasing function, right-continuous with left limits, with $A(0) = 0$ and $A(t) \to +\infty$ as $t \to +\infty$, called a time-scaling function (the name is taken from Cha and Mercier, 2019, Example 1), and let Y be a Levy process. The process

$$X_t = Y_{A(t)}, \qquad t \geq 0,$$

is a time-scaled Levy process (Kahle et al., 2016, p. xix-xx; Cha and Mercier, 2019, Example 1, for the Wiener process): $X_0 = 0$, its increments are independent and $X_t - X_s$ has the distribution of $Y_{A(t)-A(s)}$. Its increments are not stationary when A is not linear: X is not a Levy process, but a non-homogeneous Levy process. When A is continuous, X is also stochastically continuous: it is an additive process in the sense of Sato (2001, p. 3), that is a process satisfying the conditions 1, 2, 4 and 5 above.

The time-scaling function in the package. The function A is given as the argument `A` of the functions `sim_gamma()`, `sim_wiener()` and `sim_ig()`. It is a vectorized R function of `t`: `A(t)` returns one value per time, for instance with `function(t) 2 * t^1.5`. Each of these functions checks that $A(0) = 0$ and that A is finite and non-decreasing at the inspection times, and raises

an error otherwise. The conditions "right-continuous with left limits" and $A(t) \to +\infty$ cannot be checked on a finite set of times: they are left to the user. If A is constant between two inspection times, the increment of X is 0.

Identifiability. One parameter of Y (a for the gamma process, μ for the Wiener and inverse Gaussian processes) is set to 1 and contained in A. With $\Delta A = A(t) - A(s)$:

Function	Parameters	$X_t - X_s$
`sim_gamma()`	`b`, `A`	$\Gamma(\Delta A, b)$
`sim_wiener()`	`sigma`, `A`	$\mathcal{N}(\Delta A, \sigma^2 \Delta A)$
`sim_ig()`	`eta`, `A`	$IG(\Delta A, \eta \Delta A^2)$

The time-scaled Wiener process is $X_t = A(t) + \sigma B_{A(t)}$, with B a standard Brownian motion.

Linear time-scaling function. With $A(t) = at$, the time-scaled gamma process is the gamma process with shape a. With $A(t) = \mu t$, the time-scaled Wiener process $\mu t + \sigma B_{\mu t}$ has the distribution of $\mu t + \sigma\sqrt{\mu}\, B_t$, since $(B_{\mu t})_{t \geq 0}$ has the distribution of $(\sqrt{\mu}\, B_t)_{t \geq 0}$: it is the Wiener process with drift μ and volatility $\sigma\sqrt{\mu}$. In the same way, with $A(t) = \mu t$, the increments of the time-scaled inverse Gaussian process are $IG(\mu \Delta t, \eta \mu^2 \Delta t^2)$: it is the inverse Gaussian process with parameters μ and $\eta\mu^2$.

Getting started

A simulation always follows the same three steps.

1. **Choose the inspection times** `t`. Three forms are accepted by every `sim_` function:
   - a numeric vector, for instance the component `time` of the result of `inspection_times()`: all the trajectories share these times, and their number is given by `n_traj`;
   - a list of numeric vectors, one per trajectory, for instance `list(c(0, 1, 3), c(0, 2, 4, 6))`;
   - the data frame returned by `inspection_times()`, which gives one plan per trajectory, for instance with the pattern `"decreasing"` for trajectories inspected more and more often. `n_traj` must then stay 1, otherwise an error is raised: to simulate `n_traj` trajectories on the same times, give `plan$time`.

   The times of each trajectory must be finite, start at 0 and be strictly increasing, with at least two times (0 and one inspection). These conditions are checked by every `sim_` function, also for the result of `inspection_times()`, and an error explains which condition fails.
2. **Simulate** with the function of the chosen process and its parameters; for a gamma process, for instance, `sim_gamma(t, a = 2, b = 3)`. For a time-scaled Levy process, give the time-scaling function `A` (see the section Time-scaled Levy processes) instead of the fixed parameter, for instance `sim_gamma(t, b = 3, A = function(t) 2 * t^1.5)`.
3. **Store and use the result** `x`, an object of class `DegradSim`. Its form depends on the times:
   - **same times for all the trajectories**: a matrix with the column `t` (inspection times, starting at 0) and one column per trajectory, `X1`, ..., `Xn` (levels, equal to 0 at `t = 0`). The levels of the k-th trajectory are the column `x[, "Xk"]`, which is also the column number $k + 1$, `x[, k + 1]`;
   - **different times**: a list with one data frame per trajectory. The data frame `x[[k]]` of the k-th trajectory has two columns: `t` (its inspection times) and `X` (its levels). A column can be taken by its name or by its position: `x[[k]]$t` is the same as `x[[k]][, 1]` (first column), and `x[[k]]$X` is the same as `x[[k]][, 2]` (second column).

In both cases, `x` prints the model, the parameters, the time-scaling function if any and the first simulated values; `traj_df(x)` gives all the data in one data frame (`id`, `t`, `X`, and `dt`, `dX` with `increments = TRUE`) and `plot_traj(x)` plots the trajectories. See the examples below.

Use `set.seed()` before a simulation to make it reproducible.

Other functions:

- `inspection_times()`: inspection times following a chosen pattern (regular, with increasing or decreasing gaps, or alternating between frequent and rare inspections);
- `plot_plan()`: plot of the inspection times, one row per trajectory (`ggplot2`);
- `plot_traj()`: plot of the simulated trajectories (`ggplot2`);
- `traj_df()`: simulated data as a data frame.

Simulation methods

The argument `method` of the `sim_` functions chooses among the three simulation methods of Kahle et al. (2016) for the Wiener process (section 1.3) and the gamma process (section 2.5). `"RWA"` is available for all the processes, `"Bridge"` for the gamma, Wiener, inverse Gaussian, variance-gamma and Meixner processes, and `"Series"` is not currently available.

Random walk approximation (`method = "RWA"`, default). All the simulated processes have independent increments with known distributions. The values at the inspection times $0 = t_0 < t_1 < \cdots < t_n$ are therefore simulated by the random walk approximation (Kahle et al., 2016, section 1.3.1, algorithm 1.7, p. 6, for the Brownian motion, and section 2.5.1, algorithm 2.32, p. 81, for the gamma process): independent increments ΔX_i with the distribution of $X_{t_i} - X_{t_{i-1}}$ are drawn, then $X_{t_0} = 0$ and $X_{t_i} = \Delta X_1 + \cdots + \Delta X_i$. The vector $(X_{t_1}, \ldots, X_{t_n})$ has exactly the distribution of the process at the inspection times; the name "approximation" refers only to the path between two inspection times, which is not simulated. Algorithm 2.32 is written for the gamma process, but the same scheme applies to every process with independent increments. The way each increment is drawn is given in the help page of each simulation function.

Bridge sampling (`method = "Bridge"`; Kahle et al., 2016, section 1.3.2, algorithm 1.9, p. 8, for the Brownian motion, and section 2.5.2, algorithm 2.34, p. 85, for the gamma process). The value at the last inspection time is drawn first, with its distribution. The value at the middle of the list of inspection times is then drawn given its two known neighbours, with the bridge of the process, and so on in each part of the list. The bridges are the functions `levyBridge::gamma_bridge()`, `levyBridge::wiener_bridge()`, `levyBridge::ig_bridge()`, `levyBridge::vg_bridge()` and `levyBridge::meixner_bridge()`. The values at the inspection times have exactly the distribution of the process, as with `"RWA"`, but the same seed gives a different trajectory.

The algorithms of Kahle et al. (2016) divide each interval in half, for $2^n + 1$ points, and are written for the standard Brownian motion and for a gamma process of rate 1. This implementation is more general: taking the middle of the list allows any number of inspection times, the bridges include the drift, the volatility and the rate, and the same scheme is applied to the inverse Gaussian, variance-gamma and Meixner processes. For a time-scaled process, the bridge is applied at the times $A(t_i)$, and the level does not change where A is constant.

Truncated series representation (`method = "Series"`, argument `type`). The path is approximated on the whole interval $[0, T]$ by a truncated series: the Karhunen-Loève expansion of the Brownian motion (Kahle et al., 2016, section 1.3.3, p. 9, equation [1.1]), and, for the gamma process, compound Poisson process approximations obtained by truncating the series representations of Bondesson, of rejection and of thinning (section 2.5.3, algorithm 2.35, p. 87, based on theorem 2.31).

Author(s)

Maintainer: Amedée Rytrace Tito <rytracetito5@gmail.com>

Authors:

- Christian Paroissin <christian.paroissin@univ-pau.fr>

References

Levy processes and time-scaled Levy processes

Sato, K. (2001). Basic results on Lévy processes. In O. E. Barndorff-Nielsen, T. Mikosch and S. I. Resnick (eds.), *Lévy Processes: Theory and Applications*, 3-37. Birkhäuser, Boston. doi:10.1007/9781461201977

Kahle, W., Mercier, S. and Paroissin, C. (2016). *Degradation Processes in Reliability*. ISTE and Wiley, London. ISBN 978-1-84821-888-8.

Cha, J. H. and Mercier, S. (2019). Transformed Lévy processes as state-dependent wear models. *Advances in Applied Probability*, 51(2), 468-486. doi:10.1017/apr.2019.21

Processes

Gamma process: van Noortwijk, J. M. (2009). A survey of the application of gamma processes in maintenance. *Reliability Engineering & System Safety*, 94(1), 2-21. doi:10.1016/j.ress.2007.03.019

Wiener process: Whitmore, G. A. (1995). Estimating degradation by a Wiener diffusion process subject to measurement error. *Lifetime Data Analysis*, 1, 307-319.

Inverse Gaussian process: Wang, X. and Xu, D. (2010). An inverse Gaussian process model for degradation data. *Technometrics*, 52(2), 188-197. doi:10.1198/TECH.2009.08197

Variance-gamma process: Madan, D. B., Carr, P. P. and Chang, E. C. (1998). The variance gamma process and option pricing. *European Finance Review*, 2(1), 79-105. doi:10.1023/A:1009703431535

Meixner process: Schoutens, W. and Teugels, J. L. (1998). Lévy processes, polynomials and martingales. *Communications in Statistics. Stochastic Models*, 14(1-2), 335-349.

Normal inverse Gaussian process: Barndorff-Nielsen, O. E. (1997). Normal inverse Gaussian distributions and stochastic volatility modelling. *Scandinavian Journal of Statistics*, 24(1), 1-13. doi:10.1111/14679469.00045

Compound Poisson process: Kahle et al. (2016), chapter 3, above.

Hoyle, A. E. V. (2010). *Information-Based Models for Finance and Insurance*. PhD thesis, Imperial College London. arXiv:1010.0829.

Other Levy processes

Barndorff-Nielsen, O. E. (1977). Exponentially decreasing distributions for the logarithm of particle size. *Proceedings of the Royal Society of London. Series A*, 353, 401-419.

Carr, P., Geman, H., Madan, D. B. and Yor, M. (2002). The fine structure of asset returns: an empirical investigation. *The Journal of Business*, 75(2), 305-333. doi:10.1086/338705

Chambers, J. M., Mallows, C. L. and Stuck, B. W. (1976). A method for simulating stable random variables. *Journal of the American Statistical Association*, 71(354), 340-344.

Koponen, I. (1995). Analytic approach to the problem of convergence of truncated Lévy flights towards the Gaussian stochastic process. *Physical Review E*, 52, 1197-1199.

Kou, S. G. (2002). A jump-diffusion model for option pricing. *Management Science*, 48(8), 1086-1101. doi:10.1287/mnsc.48.8.1086.166

Merton, R. C. (1976). Option pricing when underlying stock returns are discontinuous. *Journal of Financial Economics*, 3(1-2), 125-144. doi:10.1016/0304405X(76)900222

Rosiński, J. (2007). Tempering stable processes. *Stochastic Processes and their Applications*, 117(6), 677-707.

Samorodnitsky, G. and Taqqu, M. S. (1994). *Stable Non-Gaussian Random Processes*. Chapman and Hall, New York.

Simulation methods

Michael, J. R., Schucany, W. R. and Haas, R. W. (1976). Generating random variates using transformations with multiple roots. *The American Statistician*, 30(2), 88-90. doi:10.1080/00031305.1976.10479147

Hörmann, W. and Leydold, J. (2003). Continuous random variate generation by fast numerical inversion. *ACM Transactions on Modeling and Computer Simulation*, 13(4), 347-362. doi:10.1145/945511.945517

See Also

Useful links:

- https://github.com/rytracetito/levy_packages/tree/main/DegradSim
- https://github.com/rytracetito/levy_packages
- Report bugs at https://github.com/rytracetito/levy_packages/issues

Examples

```
# 1. Inspection times: 10 trajectories, inspected more and more often
plan <- inspection_times("decreasing", n = 20, n_traj = 10, tmax = 100)
plot_plan(plan)

# 2. Simulation of a gamma process (a Levy process)
set.seed(1)
x <- sim_gamma(plan, a = 0.2, b = 1)

# 3. Same times for all the trajectories: a matrix (column t, then X1, ..., X10)
x
x[, "t"]      # inspection times
x[, "X1"]     # levels of trajectory 1
x[, 2]        # the same column, by its position
head(traj_df(x, increments = TRUE))
plot_traj(x)

# 4. Time-scaled gamma process with A(t) = 0.2 t^1.5
y <- sim_gamma(plan, b = 1, A = function(t) 0.2 * t^1.5)
y
plot_traj(y, "Time-scaled gamma process")

# 5. Different inspection times: a list of data frames (t, X)
z <- sim_gamma(list(c(0, 1, 3, 6), c(0, 2, 4, 6, 8)), a = 2, b = 3)
z[[2]]        # trajectory 2: data frame with columns t and X
z[[2]]$t      # its inspection times, the same as z[[2]][, 1]
z[[2]]$X      # its levels, the same as z[[2]][, 2]
z[[2]][, 2]

# 6. The same model by the two simulation methods
x_rwa <- sim_gamma(plan, a = 2, b = 3)
x_bridge <- sim_gamma(plan, a = 2, b = 3, method = "Bridge")
x_bridge
```

```
plot_traj(x_rwa, "Gamma process, method RWA")
plot_traj(x_bridge, "Gamma process, method Bridge")
```

`inspection_times` *Inspection times following a chosen pattern*

Description

Generates inspection times on $[0, t_{max}]$ for one or several trajectories, with gaps between two successive inspections that are constant, increasing, decreasing, or that vary between the beginning, the middle and the end of the period. The result can be given directly to the simulation functions, such as `sim_gamma()`, and plotted with `plot_plan()`.

Usage

```
inspection_times(
  pattern = "regular",
  n = 10,
  n_traj = 1,
  tmax = 100,
  digits = NULL,
  a = 0.5
)
```

Arguments

`pattern`	Pattern of the inspection times (see Details): • `"regular"` (default), `"decreasing"` or `"increasing"`; • `"dec-inc"` or `"inc-dec"`; • or a strictly increasing R function on $[0, 1]$.
`n`	Number of inspections after time 0, a positive integer. A single value gives the same number of inspections to all the trajectories; a vector of length `n_traj` gives one number per trajectory.
`n_traj`	Number of trajectories, a positive integer.
`tmax`	Time of the last inspection, $t_{max} > 0$. A single value gives the same last time to all the trajectories; a vector of length `n_traj` gives one value per trajectory.
`digits`	Number of decimal places of the times, a non-negative integer, or `NULL` (default) for no rounding. Times made equal by the rounding are removed, with a warning (see Details).
`a`	Amplitude of the patterns `"dec-inc"` and `"inc-dec"`, in $[0, 1[$: it sets the contrast between frequent and rare inspections, from 0 (regular inspections) to 1 excluded; default 0.5. Not used by the other patterns.

Details

Construction. The times are the values of a strictly increasing function f on the regular grid $x_i = i/n$, $i = 0, \ldots, n$, of $[0, 1]$, rescaled to $[0, t_{max}]$:

$$t_i = t_{max} \frac{f(x_i) - f(0)}{f(1) - f(0)}, \qquad i = 0, \ldots, n.$$

So $t_0 = 0$, $t_n = t_{max}$, and the gap between two successive inspections is about proportional to the slope $f'(x_i)$: the inspections are frequent where f is flat and rare where f is steep.

Patterns.

- `"regular"`: $f(x) = x$, constant gaps t_{max}/n;
- `"decreasing"`: $f(x) = \ln(1 + 10x)$, decreasing gaps: inspections rare at the beginning and more and more frequent; the first gap is about $f'(0)/f'(1) = 11$ times the last one;
- `"increasing"`: $f(x) = -\ln(11 - 10x)$, increasing gaps: inspections frequent at the beginning and rarer and rarer, mirror image of `"decreasing"`;
- `"dec-inc"`: $f(x) = x + \frac{a}{2\pi}\sin(2\pi x)$, gaps that decrease then increase: inspections rare at the beginning, frequent in the middle, rare at the end;
- `"inc-dec"`: $f(x) = x - \frac{a}{2\pi}\sin(2\pi x)$, gaps that increase then decrease: inspections frequent at the beginning, rare in the middle, frequent at the end.

The names describe the gaps between successive inspections: `"decreasing"` means gaps that decrease, so more and more frequent inspections; `"dec-inc"` means gaps that decrease then increase.

Amplitude `a` of `"dec-inc"` and `"inc-dec"`. Here $f'(x) = 1 \pm a\cos(2\pi x)$: the gaps vary between $1 - a$ and $1 + a$ times the mean gap t_{max}/n, so that the largest gap is $(1 + a)/(1 - a)$ times the smallest one. With the default $a = 0.5$, this ratio is 3 and the gaps change progressively; with $a = 0.8$, it is 9 and the frequent inspections gather in a short time; $a = 0$ gives regular inspections.

Rounding. With `digits`, the times are rounded to `digits` decimal places. If the rounding makes two successive times equal, the repeated times are removed, so that the trajectory has fewer than `n` inspections after time 0, and a warning gives the number of inspections that are kept.

Other patterns. Any strictly increasing R function on $[0, 1]$ can be given as `pattern`, for instance `sqrt`; an error is raised if it is not strictly increasing on the grid.

Value

A data frame with one row per inspection and two columns: `id` (number of the trajectory, 1 to `n_traj`) and `time` (inspection time). Each trajectory starts at time 0 and ends at its `tmax`, with strictly increasing times. The times of the trajectory k are `plan$time[plan$id == k]`.

See Also

`plot_plan()` to plot the inspection times, `sim_gamma()` and the other simulation functions, which accept the result as inspection times.

Examples

```
# 1. Regular inspections: 20 inspections after time 0, up to 100
inspection_times("regular", n = 20, tmax = 100)

# 2. More and more frequent inspections, times rounded to integers
inspection_times("decreasing", n = 20, tmax = 100, digits = 0)

# 3. Fewer and fewer inspections
inspection_times("increasing", n = 20, tmax = 100, digits = 0)

# 4. Rare at the beginning, frequent in the middle, rare at the end
inspection_times("dec-inc", n = 20, tmax = 100, digits = 1)

# 5. Frequent at the beginning and at the end, rare in the middle
```

```
inspection_times("inc-dec", n = 20, tmax = 100, digits = 1)

# 6. Stronger contrast between frequent and rare inspections
inspection_times("inc-dec", n = 20, tmax = 100, digits = 1, a = 0.8)

# Gaps between inspections: from about 2.5 to 7.5 with the default amplitude
range(diff(inspection_times("inc-dec", n = 20, tmax = 100)$time))

# 7. Three trajectories with 10, 15 and 20 inspections, up to 50, 100 and 150
plan <- inspection_times("decreasing", n = c(10, 15, 20), n_traj = 3,
                         tmax = c(50, 100, 150), digits = 1)
plan

# Times of the second trajectory only
plan$time[plan$id == 2]

# 8. Same number of inspections and same tmax: the same times for the
#    three trajectories
inspection_times("decreasing", n = 20, n_traj = 3, tmax = 100)

# 9. Rounding too strong: the repeated times are removed with a warning
inspection_times("increasing", n = 30, tmax = 20, digits = 0)

# 10. Any strictly increasing function on [0, 1]: with sqrt, the
#     inspections are rare at the beginning and more and more frequent
inspection_times(sqrt, n = 20, tmax = 100, digits = 0)

# 11. Mixture of the increasing and decreasing patterns: frequent
#     inspections around a point that moves to the right when w increases
#     (0 < w < 1)
w <- 0.8
f <- function(x) w * log(1 + 10 * x) - (1 - w) * log(11 - 10 * x)
inspection_times(f, n = 20, tmax = 100, digits = 1)

# 12. Plot the inspection times, one row per trajectory, then simulate
plan <- inspection_times("inc-dec", n = c(10, 20, 30), n_traj = 3, tmax = 100)
plot_plan(plan)
x <- sim_gamma(plan, a = 0.2, b = 1)
plot_traj(x)
```

`plot_plan` *Plot inspection times*

Description

Draws the inspection times of one or several trajectories with `ggplot2`: one point per inspection, and one row per trajectory when there are several. It shows at a glance where the inspections are frequent and where they are rare.

Usage

```
plot_plan(t, title = "Inspection times")
```

Arguments

t	Inspection times, in one of the three forms accepted by the sim_ functions: • a numeric vector, for one trajectory, for instance the component time of the result of inspection_times(); • a list of numeric vectors, one per trajectory, for instance list(c(0, 1, 3), c(0, 2, 4, 6)); • the result of inspection_times(), one plan per trajectory. The times of each trajectory start at 0 and are strictly increasing.
title	Title of the plot, a character string; by default "Inspection times".

Value

A ggplot object. It is displayed when printed, and it can be modified with the usual ggplot2 functions, added with +: for instance + ggplot2::labs(x = "Time (hours)") to change the axis label.

See Also

inspection_times(), plot_traj()

Examples

```
# One trajectory with 20 regular inspections up to 100 (step 5)
plot_plan(inspection_times("regular", n = 20, tmax = 100)$time)

# Inspections more and more frequent over time, with a custom title
t <- inspection_times("decreasing", n = 20, tmax = 100)
plot_plan(t, "Decreasing gaps between inspections")

# Five trajectories, each with its own times: one row per trajectory
plan <- inspection_times("inc-dec", n = c(10, 15, 20, 25, 30), n_traj = 5, tmax = 100)
plot_plan(plan)

# Different inspection times given as a list
plot_plan(list(c(0, 1, 3, 6), c(0, 2, 4, 6, 8, 10)))

# Change the axis label
plot_plan(t) + ggplot2::labs(x = "Time (hours)")
```

---

plot_traj *Plot simulated trajectories*

Description

Draws the trajectories simulated by a sim_ function with ggplot2: one grey line per trajectory, with a point at each inspection.

Usage

```
plot_traj(x, title = NULL)
```

Arguments

x	Result of a simulation function of the package, for instance sim_gamma().
title	Title of the plot, a character string; by default the name of the process, for instance "gamma process".

Value

A ggplot object. It is displayed when printed, and it can be modified with the usual ggplot2 functions, added with +: for instance + ggplot2::geom_hline(yintercept = 10) to draw a failure threshold, or + ggplot2::labs(x = "Time (hours)") to change an axis label.

See Also

traj_df()

Examples

```
set.seed(1)
t <- inspection_times("regular", n = 20, tmax = 100)$time
x <- sim_wiener(t, mu = 1.5, sigma = 2, n_traj = 5)
plot_traj(x)

# Custom title
plot_traj(x, "Five trajectories of a Wiener process")

# Add a failure threshold and change the axis labels
plot_traj(x) +
  ggplot2::geom_hline(yintercept = 100, linetype = "dashed", colour = "red") +
  ggplot2::labs(x = "Time (hours)", y = "Degradation (%)")
```

---

sim_cpoisson *Simulation of the compound Poisson process*

Description

Simulates exactly, at given inspection times, one or several trajectories of a compound Poisson process, a Levy process: a shock model where each shock adds a random amount of degradation.

Usage

```
sim_cpoisson(
  t,
  lambda,
  rjump = function(n) stats::rexp(n, 1),
  method = "RWA",
  type = NULL,
  n_traj = 1
)
```

Arguments

`t`	Inspection times. The times of each trajectory must start at 0 and be strictly increasing. Three forms are accepted: • a numeric vector: all the trajectories share these times, and their number is `n_traj`. The column `time` of the result of `inspection_times()` can be used; • a list of vectors, one per trajectory, for instance `list(c(0, 1, 3), c(0, 2, 4, 6))`: each trajectory has its own times (`n_traj` must then stay 1); • the data frame returned by `inspection_times()`, which gives one plan per trajectory (`n_traj` must then stay 1).
`lambda`	Intensity of the shocks, $\lambda > 0$: mean number of shocks per unit of time.
`rjump`	Distribution of the jumps: an R function of `n` that returns `n` independent jumps, for instance `function(n) rgamma(n, 2, 3)`. The default `function(n) rexp(n, 1)` gives exponential jumps of mean 1, and `function(n) rep(1, n)` gives the Poisson process.
`method`	Simulation method: only `"RWA"` (random walk approximation, default) is available for this process. `"Bridge"` gives an error, because the package `levyBridge` has no bridge for this process, and `"Series"` is not currently available.
`type`	Used only with `method = "Series"`, to choose the series representation; `NULL` (default).
`n_traj`	Number of trajectories to simulate when `t` is a vector (default 1), a positive integer. It must stay 1 when `t` is a plan or a list, which already gives the trajectories: to simulate several trajectories on the times of a plan, give `plan$time`.

Value

An object of class `DegradSim`: a matrix (column `t`, then one column of levels per trajectory) when all the trajectories have the same inspection times, otherwise a list with one data frame (`t`, `X`) per trajectory. The model, the parameters and the time-scaling function, if any, are stored as attributes. The use of the result (a trajectory, its times and levels, `traj_df()`, `plot_traj()`) is described in the section Getting started of DegradSim-package.

Model

Let N be a Poisson process with intensity $\lambda > 0$ (the times of the shocks) and $J_1, J_2, \ldots$ independent random variables with the same distribution, independent of N (the sizes of the shocks). The compound Poisson process is

$$X_t = \sum_{k=1}^{N_t} J_k, \qquad t \geq 0,$$

with $X_t = 0$ when $N_t = 0$. It is a Levy process: $X_0 = 0$, and for $0 \leq s < t$, $X_t - X_s$ is the sum of $N_t - N_s \sim Poisson(\lambda(t-s))$ independent jumps. Hence $E[X_t] = \lambda t\, E[J]$ and $Var(X_t) = \lambda t\, E[J^2]$. The trajectories are piecewise constant, with a finite number of jumps on every interval: an increment is 0 with probability at least $e^{-\lambda(t-s)}$ (no shock). The process is non-decreasing when the jumps are non-negative, which is the usual case for a degradation (cumulative shock model, Kahle et al., 2016, chapter 3). With jumps equal to 1, it is the Poisson process N itself.

Exact simulation

Random walk approximation, described in DegradSim-package: with $\Delta t_i = t_i - t_{i-1}$, the increments are drawn independently, then cumulated:

1. draw the numbers of shocks $N_i \sim Poisson(\lambda \Delta t_i)$ with `stats::rpois()`;
2. draw $N_1 + \cdots + N_n$ jumps with `rjump`;
3. compute the increment ΔX_i as the sum of the N_i jumps of the interval $]t_{i-1}, t_i]$ (0 if $N_i = 0$).

References

Kahle, W., Mercier, S. and Paroissin, C. (2016). *Degradation Processes in Reliability*. ISTE and Wiley, London. ISBN 978-1-84821-888-8.

Hoyle, A. E. V. (2010). *Information-Based Models for Finance and Insurance*. PhD thesis, Imperial College London. arXiv:1010.0829.

Sato, K. (2001). Basic results on Lévy processes. In O. E. Barndorff-Nielsen, T. Mikosch and S. I. Resnick (eds.), *Lévy Processes: Theory and Applications*, 3-37. Birkhäuser, Boston. doi:10.1007/9781461201977

See Also

`sim_gamma()`, `plot_traj()`, `traj_df()`, `inspection_times()`, `plot_plan()`

Examples

```
# 1. One trajectory with 20 regular inspections up to 100 (step 5), exponential
#    jumps of mean 1 (default)
t <- inspection_times("regular", n = 20, tmax = 100)$time
set.seed(1)
x <- sim_cpoisson(t, lambda = 2)
x
plot_traj(x)

# 1b. One trajectory with 20 inspections up to 100 whose steps increase
#     (inspections less and less frequent)
t_inc <- inspection_times("increasing", n = 20, tmax = 100)$time
round(diff(t_inc), 2)
x <- sim_cpoisson(t_inc, lambda = 2)
plot_traj(x)

# 2. Ten trajectories with gamma jumps of mean 2/3
x <- sim_cpoisson(t, lambda = 2, rjump = function(n) rgamma(n, 2, 3), n_traj = 10)
plot_traj(x)

# Same times for all the trajectories: a matrix with the column t,
# then one column per trajectory (X1, X2, ...)
head(x[, 1:4])

# 3. Rare and large shocks, then frequent and small shocks: same mean
x1 <- sim_cpoisson(t, lambda = 0.2, rjump = function(n) rexp(n, 0.1), n_traj = 10)
x2 <- sim_cpoisson(t, lambda = 20, rjump = function(n) rexp(n, 10), n_traj = 10)
plot_traj(x1, "Rare and large shocks")
plot_traj(x2, "Frequent and small shocks")

# 4. The Poisson process: jumps equal to 1
```

```
x <- sim_cpoisson(t, lambda = 2, rjump = function(n) rep(1, n), n_traj = 10)
plot_traj(x, "Poisson process")

# 5. Irregular inspections, more and more frequent over time
plan <- inspection_times("decreasing", n = 20, n_traj = 5, tmax = 100)
x <- sim_cpoisson(plan, lambda = 2)
plot_traj(x)

# 6. Different inspection times for each trajectory: a list of data frames
x <- sim_cpoisson(list(c(0, 1, 3, 6), c(0, 2, 4, 6, 8, 10)), lambda = 2)
x[[2]]
x[[2]]$X

# 7. All the simulated data in a single data frame, with the increments
head(traj_df(x))
head(traj_df(x, increments = TRUE))

# 8. Increments over the constant step h = 5, with lambda = 0.2 and exponential
#    jumps of rate 1: an increment is 0 with probability exp(-lambda h) (no shock);
#    otherwise its density is sum_k P(N = k) dgamma(y, k, 1) / (1 - exp(-lambda h)),
#    where N has the Poisson distribution of mean lambda h = 1
t8 <- inspection_times("regular", n = 20, tmax = 100)$time
x <- sim_cpoisson(t8, lambda = 0.2, n_traj = 100)
dx <- traj_df(x, increments = TRUE)$dX
dx <- dx[!is.na(dx)]
c(zero_observed = mean(dx == 0), zero_theory = exp(-1))
dpos <- function(y) {
  vapply(y, function(v) sum(dpois(1:30, 1) * dgamma(v, 1:30, 1)), 0) / (1 - exp(-1))
}
ggplot2::ggplot(data.frame(dx = dx[dx > 0]), ggplot2::aes(dx)) +
  ggplot2::geom_histogram(ggplot2::aes(y = ggplot2::after_stat(density)),
                          bins = 30, boundary = 0, fill = "grey80", colour = "grey40") +
  ggplot2::stat_function(fun = dpos, colour = "red") +
 ggplot2::labs(x = "dX > 0", y = "Density", title = "Non-zero increments, compound Poisson")
```

`sim_gamma` *Simulation of the gamma process*

Description

Simulates exactly, at given inspection times, one or several trajectories of a gamma process (a Levy process) or of a time-scaled gamma process, obtained with a time-scaling function A.

Usage

```
sim_gamma(t, a, b, A = NULL, method = "RWA", type = NULL, n_traj = 1)
```

Arguments

`t`	Inspection times. The times of each trajectory must start at 0 and be strictly increasing. Three forms are accepted: • a numeric vector: all the trajectories share these times, and their number is `n_traj`. The column `time` of the result of `inspection_times()` can be used;

- a list of vectors, one per trajectory, for instance `list(c(0, 1, 3), c(0, 2, 4, 6))`: each trajectory has its own times (`n_traj` must then stay 1);
- the data frame returned by `inspection_times()`, which gives one plan per trajectory (`n_traj` must then stay 1).

`a` Shape parameter of the gamma process, $a > 0$: the increment over a time Δt has shape $a\Delta t$. Not given when `A` is given (the shape is then $A(t) - A(s)$).

`b` Rate parameter, $b > 0$ (the scale is $1/b$).

`A` `NULL` (default) for the gamma process, or the time-scaling function A of the time-scaled process (defined in DegradSim-package): a vectorized R function of `t`, for instance `function(t) 2 * t^1.5`.

`method` Simulation method, described in the section Simulation methods of DegradSim-package: `"RWA"` (random walk approximation, default) or `"Bridge"` (bridge sampling). The two give the same distribution, but different trajectories with the same seed. `"Series"` (truncated series representation) is not currently available and gives an error.

`type` Used only with `method = "Series"`, to choose the series representation; `NULL` (default).

`n_traj` Number of trajectories to simulate when `t` is a vector (default 1), a positive integer. It must stay 1 when `t` is a plan or a list, which already gives the trajectories: to simulate several trajectories on the times of a plan, give `plan$time`.

Value

An object of class `DegradSim`: a matrix (column `t`, then one column of levels per trajectory) when all the trajectories have the same inspection times, otherwise a list with one data frame (`t`, `X`) per trajectory. The model, the parameters and the time-scaling function, if any, are stored as attributes. The use of the result (a trajectory, its times and levels, `traj_df()`, `plot_traj()`) is described in the section Getting started of DegradSim-package.

Model

$\Gamma(k, b)$ denotes the gamma distribution with shape $k > 0$ and **rate** $b > 0$ (scale $1/b$), of density $b^k x^{k-1} e^{-bx}/\Gamma(k)$, $x > 0$.

Gamma process. Let $a > 0$ and $b > 0$. The process satisfies $X_0 = 0$, has independent increments, and for $0 \le s < t$,

$$X_t - X_s \sim \Gamma\big(a(t-s),\ b\big).$$

Then $E[X_t] = at/b$ and $Var(X_t) = at/b^2$. It is a Levy process, increasing and purely made of jumps.

Time-scaled gamma process (`A` given). With the time-scaling function A defined in DegradSim-package, $X_t = Y_{A(t)}$, where Y is the gamma process with $a = 1$ (identifiability, see DegradSim-package) and rate b:

$$X_t - X_s \sim \Gamma\big(A(t) - A(s),\ b\big),$$

$E[X_t] = A(t)/b$ and $Var(X_t) = A(t)/b^2$. This is the gamma process with shape function A of Kahle et al. (2016, chapter 2) and of Cha and Mercier (2019, Example 2). A classical choice is $A(t) = c\,t^\beta$.

Exact simulation

Random walk approximation, described in DegradSim-package: the increments $\Delta X_i \sim \Gamma(\Delta_i, b)$, with $\Delta_i = a(t_i - t_{i-1})$ or $\Delta_i = A(t_i) - A(t_{i-1})$, are drawn independently with `stats::rgamma()`, then cumulated.

With `method = "Bridge"`, the values are drawn by `levyBridge::gamma_bridge()`: see the section Simulation methods of DegradSim-package.

References

Kahle, W., Mercier, S. and Paroissin, C. (2016). *Degradation Processes in Reliability*. ISTE and Wiley, London. ISBN 978-1-84821-888-8.

Cha, J. H. and Mercier, S. (2019). Transformed Lévy processes as state-dependent wear models. *Advances in Applied Probability*, 51(2), 468-486. doi:10.1017/apr.2019.21

van Noortwijk, J. M. (2009). A survey of the application of gamma processes in maintenance. *Reliability Engineering & System Safety*, 94(1), 2-21. doi:10.1016/j.ress.2007.03.019

See Also

- `sim_ig()`, `sim_wiener()`: the other time-scaled processes;
- `plot_traj()`, `traj_df()`, `inspection_times()`, `plot_plan()`.

Examples

```
# 1. One trajectory with 20 regular inspections up to 100 (step 5)
t <- inspection_times("regular", n = 20, tmax = 100)$time
set.seed(1)
x <- sim_gamma(t, a = 2, b = 3)
x
plot_traj(x)

# 1b. One trajectory with 20 inspections up to 100 whose steps increase
#     (inspections less and less frequent)
t_inc <- inspection_times("increasing", n = 20, tmax = 100)$time
round(diff(t_inc), 2)
x <- sim_gamma(t_inc, a = 2, b = 3)
plot_traj(x)

# 2. Ten trajectories with the same inspection times
x <- sim_gamma(t, a = 2, b = 3, n_traj = 10)
plot_traj(x)

# Same times for all the trajectories: a matrix with the column t,
# then one column per trajectory (X1, X2, ...)
head(x[, 1:4])

# 3. Irregular inspections, more and more frequent over time
plan <- inspection_times("decreasing", n = 20, n_traj = 5, tmax = 100)
x <- sim_gamma(plan, a = 2, b = 3)
plot_traj(x)

# 4. Different inspection times for each trajectory: a list of data frames
x <- sim_gamma(list(c(0, 1, 3, 6), c(0, 2, 4, 6, 8, 10)), a = 2, b = 3)
x[[2]]
x[[2]]$X
```

```
# 5. Time-scaled gamma process (model of Cha and Mercier, 2019, Example 2):
#    X_t - X_s ~ Gamma(A(t) - A(s), b), here with A(t) = 2 t^1.5
x <- sim_gamma(t, b = 3, A = function(t) 2 * t^1.5, n_traj = 10)
x
plot_traj(x, "Time-scaled gamma process")

# 6. With A(t) = a t, the time-scaled gamma process is the gamma process
#    with shape a
set.seed(2)
y1 <- sim_gamma(t, b = 3, A = function(t) 2 * t)
set.seed(2)
y2 <- sim_gamma(t, a = 2, b = 3)
all.equal(unclass(y1)[, 2], unclass(y2)[, 2])

# 7. All the simulated data in a single data frame, with the increments
head(traj_df(x))
head(traj_df(x, increments = TRUE))

# 8. Histogram of the increments, with their density in red. Constant
#    step h = 5: the increments are i.i.d. Gamma(a h, b) = Gamma(10, 3)
t8 <- inspection_times("regular", n = 20, tmax = 100)$time
x <- sim_gamma(t8, a = 2, b = 3, n_traj = 100)
dx <- traj_df(x, increments = TRUE)$dX
dx <- dx[!is.na(dx)]
ggplot2::ggplot(data.frame(dx), ggplot2::aes(dx)) +
  ggplot2::geom_histogram(ggplot2::aes(y = ggplot2::after_stat(density)),
                          bins = 30, boundary = 0, fill = "grey80", colour = "grey40") +
  ggplot2::stat_function(fun = function(y) dgamma(y, shape = 2 * 5, rate = 3),
                         colour = "red") +
  ggplot2::labs(x = "dX", y = "Density", title = "Increments of the gamma process")

# 9. Bridge sampling: one trajectory. The printed result gives the method
x <- sim_gamma(t, a = 2, b = 3, method = "Bridge")
x
plot_traj(x)

# 10. Bridge sampling: ten trajectories with the same inspection times
x <- sim_gamma(t, a = 2, b = 3, n_traj = 10, method = "Bridge")
plot_traj(x)
head(x[, 1:4])

# 11. Bridge sampling: irregular inspections, the same as above
x <- sim_gamma(plan, a = 2, b = 3, method = "Bridge")
plot_traj(x)

# 12. Bridge sampling: all the simulated data in a single data frame, with the increments
head(traj_df(x, increments = TRUE))

# 13. Bridge sampling: different inspection times for each trajectory
x <- sim_gamma(list(c(0, 1, 3, 6), c(0, 2, 4, 6, 8, 10)), a = 2, b = 3,
               method = "Bridge")
x[[2]]

# 14. Bridge sampling: histogram of the increments, with the same density as above
x <- sim_gamma(t8, a = 2, b = 3, n_traj = 100, method = "Bridge")
dx <- traj_df(x, increments = TRUE)$dX
```

```
dx <- dx[!is.na(dx)]
ggplot2::ggplot(data.frame(dx), ggplot2::aes(dx)) +
  ggplot2::geom_histogram(ggplot2::aes(y = ggplot2::after_stat(density)),
                          bins = 30, boundary = 0, fill = "grey80", colour = "grey40") +
  ggplot2::stat_function(fun = function(y) dgamma(y, shape = 2 * 5, rate = 3),
                         colour = "red") +
  ggplot2::labs(x = "dX", y = "Density", title = "Increments of the gamma process, bridge")

# 15. Bridge sampling: time-scaled process with A(t) = 0.2 t^1.5
x <- sim_gamma(t8, b = 3, A = function(t) 0.2 * t^1.5, method = "Bridge", n_traj = 5)
plot_traj(x)
```

`sim_ig` *Simulation of the inverse Gaussian process*

Description

Simulates exactly, at given inspection times, one or several trajectories of an inverse Gaussian (IG) process (a Levy process) or of a time-scaled IG process, obtained with a time-scaling function A.

Usage

```
sim_ig(t, mu, eta, A = NULL, method = "RWA", type = NULL, n_traj = 1)
```

Arguments

`t`	Inspection times. The times of each trajectory must start at 0 and be strictly increasing. Three forms are accepted: • a numeric vector: all the trajectories share these times, and their number is `n_traj`. The column `time` of the result of `inspection_times()` can be used; • a list of vectors, one per trajectory, for instance `list(c(0, 1, 3), c(0, 2, 4, 6))`: each trajectory has its own times (`n_traj` must then stay 1); • the data frame returned by `inspection_times()`, which gives one plan per trajectory (`n_traj` must then stay 1).
`mu`	Mean degradation per unit of time of the IG process, $\mu > 0$. Not given when `A` is given (the mean of X_t is then $A(t)$).
`eta`	Shape parameter, $\eta > 0$: the larger η, the less variable the increments. The variance of X_t is $\mu^3 t/\eta$ (IG process) or $A(t)/\eta$ (time-scaled IG process).
`A`	`NULL` (default) for the gamma process, or the time-scaling function A of the time-scaled process (defined in DegradSim-package): a vectorized R function of `t`, for instance `function(t) 2 * t^1.5`.
`method`	Simulation method, described in the section Simulation methods of DegradSim-package: `"RWA"` (random walk approximation, default) or `"Bridge"` (bridge sampling). The two give the same distribution, but different trajectories with the same seed. `"Series"` (truncated series representation) is not currently available and gives an error.
`type`	Used only with `method = "Series"`, to choose the series representation; `NULL` (default).
`n_traj`	Number of trajectories to simulate when `t` is a vector (default 1), a positive integer. It must stay 1 when `t` is a plan or a list, which already gives the trajectories: to simulate several trajectories on the times of a plan, give `plan$time`.

Value

An object of class `DegradSim`: a matrix (column `t`, then one column of levels per trajectory) when all the trajectories have the same inspection times, otherwise a list with one data frame (`t`, `X`) per trajectory. The model, the parameters and the time-scaling function, if any, are stored as attributes. The use of the result (a trajectory, its times and levels, `traj_df()`, `plot_traj()`) is described in the section Getting started of DegradSim-package.

Model

$IG(m, \lambda)$ denotes the inverse Gaussian distribution with mean $m > 0$ and shape $\lambda > 0$, of density

$$\sqrt{\frac{\lambda}{2\pi x^3}} \exp\Big(-\frac{\lambda(x-m)^2}{2m^2 x}\Big), \qquad x > 0,$$

and variance m^3/λ.

IG process. Let $\mu > 0$ and $\eta > 0$. The process satisfies $X_0 = 0$, has independent increments, and for $0 \le s < t$,

$$X_t - X_s \sim IG\big(\mu(t-s),\ \eta(t-s)^2\big),$$

with the parametrization of Wang and Xu (2010). Then $E[X_t] = \mu t$ and $Var(X_t) = \mu^3 t/\eta$. It is a Levy process, increasing and purely made of jumps.

Time-scaled IG process (`A` given). With the time-scaling function A defined in DegradSim-package, $X_t = Y_{A(t)}$, where Y is the IG process with $\mu = 1$ (identifiability, see DegradSim-package):

$$X_t - X_s \sim IG\big(A(t) - A(s),\ \eta\,(A(t) - A(s))^2\big),$$

$E[X_t] = A(t)$ and $Var(X_t) = A(t)/\eta$. Cha and Mercier (2019, Example 3) consider the inverse Gaussian process with mean function A, but write its increments with a constant shape b. The form above is used here because its variances add over successive intervals.

Exact simulation

Random walk approximation, described in DegradSim-package: the increments $\Delta X_i \sim IG(m_i, \lambda_i)$, with $m_i = \mu\Delta t_i$, $\lambda_i = \eta\Delta t_i^2$ (IG process) or $m_i = \Delta A_i$, $\lambda_i = \eta\Delta A_i^2$ (time-scaled IG process), are drawn independently, then cumulated. Each increment $IG(m, \lambda)$ is drawn by the method of Michael, Schucany and Haas (1976), based on $\lambda(X-m)^2/(m^2 X) \sim \chi^2_1$: draw $Z \sim \mathcal{N}(0,1)$, set $\nu = Z^2$, $x_1 = m + m^2\nu/(2\lambda) - m\sqrt{4m\lambda\nu + m^2\nu^2}/(2\lambda)$, and return x_1 with probability $m/(m + x_1)$, otherwise m^2/x_1.

With `method = "Bridge"`, the values are drawn by `levyBridge::ig_bridge()`: see the section Simulation methods of DegradSim-package.

References

Cha, J. H. and Mercier, S. (2019). Transformed Lévy processes as state-dependent wear models. *Advances in Applied Probability*, 51(2), 468-486. doi:10.1017/apr.2019.21

Michael, J. R., Schucany, W. R. and Haas, R. W. (1976). Generating random variates using transformations with multiple roots. *The American Statistician*, 30(2), 88-90. doi:10.1080/00031305.1976.10479147

Wang, X. and Xu, D. (2010). An inverse Gaussian process model for degradation data. *Technometrics*, 52(2), 188-197. doi:10.1198/TECH.2009.08197

See Also

- sim_gamma(), sim_wiener(): the other time-scaled processes;
- plot_traj(), traj_df(), inspection_times(), plot_plan().

Examples

```
# 1. One trajectory with 20 regular inspections up to 100 (step 5)
t <- inspection_times("regular", n = 20, tmax = 100)$time
set.seed(1)
x <- sim_ig(t, mu = 0.5, eta = 2)
x
plot_traj(x)

# 1b. One trajectory with 20 inspections up to 100 whose steps increase
#     (inspections less and less frequent)
t_inc <- inspection_times("increasing", n = 20, tmax = 100)$time
round(diff(t_inc), 2)
x <- sim_ig(t_inc, mu = 0.5, eta = 2)
plot_traj(x)

# 2. Ten trajectories with the same inspection times
x <- sim_ig(t, mu = 0.5, eta = 2, n_traj = 10)
plot_traj(x)

# Same times for all the trajectories: a matrix with the column t,
# then one column per trajectory (X1, X2, ...)
head(x[, 1:4])

# 3. Irregular inspections, less and less frequent over time
plan <- inspection_times("increasing", n = 20, n_traj = 5, tmax = 100)
x <- sim_ig(plan, mu = 0.5, eta = 2)
plot_traj(x)

# 4. Different inspection times for each trajectory: a list of data frames
x <- sim_ig(list(c(0, 1, 3, 6), c(0, 2, 4, 6, 8, 10)), mu = 0.5, eta = 2)
x[[2]]
x[[2]]$X

# 5. Time-scaled IG process with mean function A(t) = 0.5 t^1.5
#    X_t - X_s ~ IG(dA, eta dA^2)
x <- sim_ig(t, eta = 2, A = function(t) 0.5 * t^1.5, n_traj = 10)
x
plot_traj(x, "Time-scaled inverse Gaussian process")

# 6. With A(t) = mu t, the time-scaled IG process with eta is the IG
#    process with parameters mu and eta mu^2
set.seed(2)
y1 <- sim_ig(t, eta = 2, A = function(t) 0.5 * t)
set.seed(2)
y2 <- sim_ig(t, mu = 0.5, eta = 2 * 0.5^2)
all.equal(unclass(y1)[, 2], unclass(y2)[, 2])

# 7. All the simulated data in a single data frame, with the increments
head(traj_df(x))
head(traj_df(x, increments = TRUE))
```

```
# 8. Histogram of the increments, with the density of the increments in red.
#    Constant step h = 5: the increments are i.i.d.
#    with distribution IG(m, lambda), m = mu h and lambda = eta h^2
t8 <- inspection_times("regular", n = 20, tmax = 100)$time
x <- sim_ig(t8, mu = 0.5, eta = 2, n_traj = 100)
dx <- traj_df(x, increments = TRUE)$dX
dx <- dx[!is.na(dx)]
m <- 0.5 * 5
lambda <- 2 * 5^2
dig <- function(y) sqrt(lambda / (2 * pi * y^3)) * exp(-lambda * (y - m)^2 / (2 * m^2 * y))
ggplot2::ggplot(data.frame(dx), ggplot2::aes(dx)) +
  ggplot2::geom_histogram(ggplot2::aes(y = ggplot2::after_stat(density)),
                          bins = 30, boundary = 0, fill = "grey80", colour = "grey40") +
  ggplot2::stat_function(fun = dig, colour = "red") +
  ggplot2::labs(x = "dX", y = "Density",
                title = "Increments of the inverse Gaussian process")

# 9. Bridge sampling: one trajectory. The printed result gives the method
x <- sim_ig(t, mu = 0.5, eta = 2, method = "Bridge")
x
plot_traj(x)

# 10. Bridge sampling: ten trajectories with the same inspection times
x <- sim_ig(t, mu = 0.5, eta = 2, n_traj = 10, method = "Bridge")
plot_traj(x)
head(x[, 1:4])

# 11. Bridge sampling: irregular inspections, the same as above
x <- sim_ig(plan, mu = 0.5, eta = 2, method = "Bridge")
plot_traj(x)

# 12. Bridge sampling: all the simulated data in a single data frame, with the increments
head(traj_df(x, increments = TRUE))

# 13. Bridge sampling: different inspection times for each trajectory
x <- sim_ig(list(c(0, 1, 3, 6), c(0, 2, 4, 6, 8, 10)), mu = 0.5, eta = 2,
            method = "Bridge")
x[[2]]

# 14. Bridge sampling: histogram of the increments, with the same density as above
x <- sim_ig(t8, mu = 0.5, eta = 2, n_traj = 100, method = "Bridge")
dx <- traj_df(x, increments = TRUE)$dX
dx <- dx[!is.na(dx)]
m <- 0.5 * 5
lambda <- 2 * 5^2
dig <- function(y) sqrt(lambda / (2 * pi * y^3)) * exp(-lambda * (y - m)^2 / (2 * m^2 * y))
ggplot2::ggplot(data.frame(dx), ggplot2::aes(dx)) +
  ggplot2::geom_histogram(ggplot2::aes(y = ggplot2::after_stat(density)),
                          bins = 30, boundary = 0, fill = "grey80", colour = "grey40") +
  ggplot2::stat_function(fun = dig, colour = "red") +
  ggplot2::labs(x = "dX", y = "Density",
                title = "Increments of the inverse Gaussian process, bridge")

# 15. Bridge sampling: time-scaled process with A(t) = 0.2 t^1.5
x <- sim_ig(t8, eta = 2, A = function(t) 0.2 * t^1.5, method = "Bridge", n_traj = 5)
plot_traj(x)
```

`sim_meixner` *Simulation of the Meixner process*

Description

Simulates exactly, at given inspection times, one or several trajectories of a Meixner process, a Levy process.

Usage

```
sim_meixner(t, a, b, d, m, method = "RWA", type = NULL, n_traj = 1)
```

Arguments

`t`	Inspection times. The times of each trajectory must start at 0 and be strictly increasing. Three forms are accepted: • a numeric vector: all the trajectories share these times, and their number is `n_traj`. The column `time` of the result of `inspection_times()` can be used; • a list of vectors, one per trajectory, for instance `list(c(0, 1, 3), c(0, 2, 4, 6))`: each trajectory has its own times (`n_traj` must then stay 1); • the data frame returned by `inspection_times()`, which gives one plan per trajectory (`n_traj` must then stay 1).
`a`	Scale parameter, $a > 0$.
`b`	Asymmetry parameter, $b \in]-\pi, \pi[$: $b = 0$ gives symmetric increments, $b > 0$ increments skewed to the right.
`d`	Shape parameter per unit of time, $d > 0$.
`m`	Location parameter per unit of time, real.
`method`	Simulation method, described in the section Simulation methods of DegradSim-package: `"RWA"` (random walk approximation, default) or `"Bridge"` (bridge sampling). The two give the same distribution, but different trajectories with the same seed. `"Series"` (truncated series representation) is not currently available and gives an error.
`type`	Used only with `method = "Series"`, to choose the series representation; `NULL` (default).
`n_traj`	Number of trajectories to simulate when `t` is a vector (default 1), a positive integer. It must stay 1 when `t` is a plan or a list, which already gives the trajectories: to simulate several trajectories on the times of a plan, give `plan$time`.

Value

An object of class `DegradSim`: a matrix (column `t`, then one column of levels per trajectory) when all the trajectories have the same inspection times, otherwise a list with one data frame (`t`, `X`) per trajectory. The model, the parameters and the time-scaling function, if any, are stored as attributes. The use of the result (a trajectory, its times and levels, `traj_df()`, `plot_traj()`) is described in the section Getting started of DegradSim-package.

Model

The $Meixner(a, b, d, m)$ distribution, with $a > 0$, $b \in]-\pi, \pi[$, $d > 0$ and m real, has density

$$f(x) = \frac{(2\cos(b/2))^{2d}}{2a\pi\,\Gamma(2d)} \exp\Big(\frac{b(x-m)}{a}\Big)\Big|\Gamma\Big(d + i\frac{x-m}{a}\Big)\Big|^2, \qquad x \in \mathbb{R}.$$

It is infinitely divisible: the sum of independent $Meixner(a, b, d_1, m_1)$ and $Meixner(a, b, d_2, m_2)$ variables has the $Meixner(a, b, d_1 + d_2, m_1 + m_2)$ distribution.

The Meixner process satisfies $X_0 = 0$, has independent increments, and for $0 \le s < t$,

$$X_t - X_s \sim Meixner\big(a, b, d(t-s), m(t-s)\big).$$

Hence $E[X_t] = (m + ad\tan(b/2))\,t$ and $Var(X_t) = a^2 d\,t/(2\cos^2(b/2))$. It is a Levy process whose trajectories are not monotone, purely made of jumps, of infinite variation: a is a scale parameter, b sets the asymmetry, d the shape and m the location.

Exact simulation

Random walk approximation, described in DegradSim-package: with $\Delta t_i = t_i - t_{i-1}$, the increments

$$\Delta X_i \sim Meixner(a, b, d\Delta t_i, m\Delta t_i)$$

are drawn independently, then cumulated. There is no Meixner generator in `stats`: each increment is drawn by numerical inversion of its distribution function (method PINV of the package `Runuran`, built from the exact density, with an error in u of order 10^{-10}). One generator is built for each distinct time step Δt_i and reused for all the trajectories. The simulation is therefore exact up to this numerical error.

With `method = "Bridge"`, the values are drawn by `levyBridge::meixner_bridge()`: see the section Simulation methods of DegradSim-package.

References

Schoutens, W. (2003). *Lévy Processes in Finance: Pricing Financial Derivatives*. Wiley, Chichester. ISBN 978-0-470-85156-2.

Schoutens, W. and Teugels, J. L. (1998). Lévy processes, polynomials and martingales. *Communications in Statistics. Stochastic Models*, 14(1-2), 335-349.

Hörmann, W. and Leydold, J. (2003). Continuous random variate generation by fast numerical inversion. *ACM Transactions on Modeling and Computer Simulation*, 13(4), 347-362. doi:10.1145/945511.945517

See Also

`plot_traj()`, `traj_df()`, `inspection_times()`, `plot_plan()`

Examples

```
# 1. One trajectory with 20 regular inspections up to 100 (step 5)
t <- inspection_times("regular", n = 20, tmax = 100)$time
set.seed(1)
x <- sim_meixner(t, a = 1, b = 0.3, d = 1.5, m = 0.2)
x
plot_traj(x)

# 1b. One trajectory with 20 inspections up to 100 whose steps increase
```

```
#     (inspections less and less frequent)
t_inc <- inspection_times("increasing", n = 20, tmax = 100)$time
round(diff(t_inc), 2)
x <- sim_meixner(t_inc, a = 1, b = 0.3, d = 1.5, m = 0.2)
plot_traj(x)

# 2. Ten trajectories with the same inspection times
x <- sim_meixner(t, a = 1, b = 0.3, d = 1.5, m = 0.2, n_traj = 10)
plot_traj(x)

# Same times for all the trajectories: a matrix with the column t,
# then one column per trajectory (X1, X2, ...)
head(x[, 1:4])

# 3. Irregular inspections, frequent in the middle of the period
plan <- inspection_times("dec-inc", n = 20, n_traj = 5, tmax = 100)
x <- sim_meixner(plan, a = 1, b = 0.3, d = 1.5, m = 0.2)
plot_traj(x)

# 4. Different inspection times for each trajectory: a list of data frames
x <- sim_meixner(list(c(0, 1, 3, 6), c(0, 2, 4, 6, 8, 10)), a = 1, b = 0.3, d = 1.5, m = 0.2)
x[[2]]
x[[2]]$X

# 5. Symmetric increments (b = 0) and no drift (m = 0)
x <- sim_meixner(t, a = 1, b = 0, d = 1.5, m = 0, n_traj = 10)
plot_traj(x, "Symmetric Meixner process")

# 6. All the simulated data in a single data frame, with the increments
head(traj_df(x))
head(traj_df(x, increments = TRUE))

# 7. Histogram of the increments, with the density of the increments in red.
#    Constant step h = 5: the increments are i.i.d.
#    with distribution Meixner(a, b, d h, m h), density from Runuran::ud()
t7 <- inspection_times("regular", n = 20, tmax = 100)$time
x <- sim_meixner(t7, a = 1, b = 0.3, d = 1.5, m = 0.2, n_traj = 100)
dx <- traj_df(x, increments = TRUE)$dX
dx <- dx[!is.na(dx)]
dist <- Runuran::udmeixner(alpha = 1, beta = 0.3, delta = 1.5 * 5, mu = 0.2 * 5)
ggplot2::ggplot(data.frame(dx), ggplot2::aes(dx)) +
  ggplot2::geom_histogram(ggplot2::aes(y = ggplot2::after_stat(density)),
                          bins = 30, fill = "grey80", colour = "grey40") +
  ggplot2::stat_function(fun = function(y) Runuran::ud(dist, y), colour = "red") +
  ggplot2::labs(x = "dX", y = "Density", title = "Increments of the Meixner process")

# 8. Bridge sampling: one trajectory. The printed result gives the method
x <- sim_meixner(t, a = 1, b = 0.3, d = 1.5, m = 0.2, method = "Bridge")
x
plot_traj(x)

# 9. Bridge sampling: ten trajectories with the same inspection times
x <- sim_meixner(t, a = 1, b = 0.3, d = 1.5, m = 0.2, n_traj = 10, method = "Bridge")
plot_traj(x)
head(x[, 1:4])

# 10. Bridge sampling: irregular inspections, the same as above
```

```
x <- sim_meixner(plan, a = 1, b = 0.3, d = 1.5, m = 0.2, method = "Bridge")
plot_traj(x)

# 11. Bridge sampling: all the simulated data in a single data frame, with the increments
head(traj_df(x, increments = TRUE))

# 12. Bridge sampling: different inspection times for each trajectory
x <- sim_meixner(list(c(0, 1, 3, 6), c(0, 2, 4, 6, 8, 10)), a = 1, b = 0.3, d = 1.5, m = 0.2,
                 method = "Bridge")
x[[2]]

# 13. Bridge sampling: symmetric increments (b = 0) and no drift (m = 0)
x <- sim_meixner(t, a = 1, b = 0, d = 1.5, m = 0, n_traj = 10, method = "Bridge")
plot_traj(x, "Symmetric Meixner process, bridge")

# 14. Bridge sampling: histogram of the increments, with the same density as above
x <- sim_meixner(t7, a = 1, b = 0.3, d = 1.5, m = 0.2, n_traj = 100, method = "Bridge")
dx <- traj_df(x, increments = TRUE)$dX
dx <- dx[!is.na(dx)]
dist <- Runuran::udmeixner(alpha = 1, beta = 0.3, delta = 1.5 * 5, mu = 0.2 * 5)
ggplot2::ggplot(data.frame(dx), ggplot2::aes(dx)) +
  ggplot2::geom_histogram(ggplot2::aes(y = ggplot2::after_stat(density)),
                          bins = 30, fill = "grey80", colour = "grey40") +
  ggplot2::stat_function(fun = function(y) Runuran::ud(dist, y), colour = "red") +
 ggplot2::labs(x = "dX", y = "Density", title = "Increments of the Meixner process, bridge")
```

`sim_nig` *Simulation of the normal inverse Gaussian process*

Description

Simulates exactly, at given inspection times, one or several trajectories of a normal inverse Gaussian (NIG) process, a Levy process.

Usage

```
sim_nig(t, alpha, beta, delta, mu, method = "RWA", type = NULL, n_traj = 1)
```

Arguments

`t`	Inspection times. The times of each trajectory must start at 0 and be strictly increasing. Three forms are accepted: • a numeric vector: all the trajectories share these times, and their number is `n_traj`. The column `time` of the result of `inspection_times()` can be used; • a list of vectors, one per trajectory, for instance `list(c(0, 1, 3), c(0, 2, 4, 6))`: each trajectory has its own times (`n_traj` must then stay 1); • the data frame returned by `inspection_times()`, which gives one plan per trajectory (`n_traj` must then stay 1).
`alpha`	Tail parameter, $\alpha > 0$: the larger α, the lighter the tails.
`beta`	Asymmetry parameter, $\lvert\beta\rvert < \alpha$: $\beta = 0$ gives symmetric increments, $\beta > 0$ increments skewed to the right.

`delta`	Scale parameter per unit of time, $\delta > 0$.
`mu`	Location parameter per unit of time, real.
`method`	Simulation method: only `"RWA"` (random walk approximation, default) is available for this process. `"Bridge"` gives an error, because the package `levyBridge` has no bridge for this process, and `"Series"` is not currently available.
`type`	Used only with `method = "Series"`, to choose the series representation; `NULL` (default).
`n_traj`	Number of trajectories to simulate when `t` is a vector (default 1), a positive integer. It must stay 1 when `t` is a plan or a list, which already gives the trajectories: to simulate several trajectories on the times of a plan, give `plan$time`.

Value

An object of class `DegradSim`: a matrix (column `t`, then one column of levels per trajectory) when all the trajectories have the same inspection times, otherwise a list with one data frame (`t`, `X`) per trajectory. The model, the parameters and the time-scaling function, if any, are stored as attributes. The use of the result (a trajectory, its times and levels, `traj_df()`, `plot_traj()`) is described in the section Getting started of DegradSim-package.

Model

The $NIG(\alpha, \beta, \delta, \mu)$ distribution, with $0 \le |\beta| < \alpha$, $\delta > 0$ and μ real, has density

$$f(x) = \frac{\alpha\delta\, K_1\big(\alpha\sqrt{\delta^2 + (x-\mu)^2}\big)}{\pi\sqrt{\delta^2 + (x-\mu)^2}} \exp\big(\delta\gamma + \beta(x-\mu)\big), \qquad x \in \mathbb{R},$$

where $\gamma = \sqrt{\alpha^2 - \beta^2}$ and K_1 is the modified Bessel function of the second kind (Barndorff-Nielsen, 1997). It is infinitely divisible: for independent $Y_1 \sim NIG(\alpha, \beta, \delta_1, \mu_1)$ and $Y_2 \sim NIG(\alpha, \beta, \delta_2, \mu_2)$,

$$Y_1 + Y_2 \sim NIG(\alpha, \beta, \delta_1 + \delta_2, \mu_1 + \mu_2).$$

The NIG process satisfies $X_0 = 0$, has independent increments, and for $0 \le s < t$,

$$X_t - X_s \sim NIG\big(\alpha, \beta, \delta(t-s), \mu(t-s)\big).$$

Hence $E[X_t] = (\mu + \delta\beta/\gamma)\, t$ and $Var(X_t) = \delta\alpha^2 t/\gamma^3$. It is a Levy process whose trajectories are not monotone, purely made of jumps, of infinite variation: α sets the tails, β the asymmetry, δ the scale and μ the location. It is a subordinated Brownian motion (Hoyle, 2010, section 2.10): $X_t = \mu t + \beta Z_t + W_{Z_t}$, where W is a standard Brownian motion and Z an independent inverse Gaussian process with $Z_t - Z_s \sim IG(\delta(t-s)/\gamma, \delta^2(t-s)^2)$ (mean, shape), as the variance-gamma process is a Brownian motion subordinated by a gamma process.

Exact simulation

Random walk approximation, described in DegradSim-package: with $\Delta t_i = t_i - t_{i-1}$, each increment is drawn by subordination, then the increments are cumulated:

1. draw $\Delta Z_i \sim IG(\delta\Delta t_i/\gamma, (\delta\Delta t_i)^2)$ (mean, shape) by the method of Michael, Schucany and Haas (1976), as in `sim_ig()`;
2. compute $\Delta X_i = \mu\Delta t_i + \beta\Delta Z_i + \sqrt{\Delta Z_i}\,\xi_i$, with ξ_i independent $\mathcal{N}(0,1)$.

Conditionally on ΔZ_i, the increment is $\mathcal{N}(\mu\Delta t_i + \beta\Delta Z_i, \Delta Z_i)$, and its mixture over the IG distribution of ΔZ_i is the $NIG(\alpha, \beta, \delta\Delta t_i, \mu\Delta t_i)$ distribution.

References

Barndorff-Nielsen, O. E. (1997). Normal inverse Gaussian distributions and stochastic volatility modelling. *Scandinavian Journal of Statistics*, 24(1), 1-13. doi:10.1111/14679469.00045

Hoyle, A. E. V. (2010). *Information-Based Models for Finance and Insurance*. PhD thesis, Imperial College London. arXiv:1010.0829.

Michael, J. R., Schucany, W. R. and Haas, R. W. (1976). Generating random variates using transformations with multiple roots. *The American Statistician*, 30(2), 88-90. doi:10.1080/00031305.1976.10479147

See Also

`sim_vg()`, `plot_traj()`, `traj_df()`, `inspection_times()`, `plot_plan()`

Examples

```
# 1. One trajectory with 20 regular inspections up to 100 (step 5)
t <- inspection_times("regular", n = 20, tmax = 100)$time
set.seed(1)
x <- sim_nig(t, alpha = 2, beta = 0.5, delta = 1, mu = 0.2)
x
plot_traj(x)

# 1b. One trajectory with 20 inspections up to 100 whose steps increase
#     (inspections less and less frequent)
t_inc <- inspection_times("increasing", n = 20, tmax = 100)$time
round(diff(t_inc), 2)
x <- sim_nig(t_inc, alpha = 2, beta = 0.5, delta = 1, mu = 0.2)
plot_traj(x)

# 2. Ten trajectories with the same inspection times
x <- sim_nig(t, alpha = 2, beta = 0.5, delta = 1, mu = 0.2, n_traj = 10)
plot_traj(x)

# Same times for all the trajectories: a matrix with the column t,
# then one column per trajectory (X1, X2, ...)
head(x[, 1:4])

# 3. Irregular inspections, more and more frequent over time
plan <- inspection_times("decreasing", n = 20, n_traj = 5, tmax = 100)
x <- sim_nig(plan, alpha = 2, beta = 0.5, delta = 1, mu = 0.2)
plot_traj(x)

# 4. Different inspection times for each trajectory: a list of data frames
x <- sim_nig(list(c(0, 1, 3, 6), c(0, 2, 4, 6, 8, 10)),
             alpha = 2, beta = 0.5, delta = 1, mu = 0.2)
x[[2]]
x[[2]]$X

# 5. Heavier tails (small alpha) and symmetric increments (beta = 0)
x <- sim_nig(t, alpha = 0.5, beta = 0, delta = 1, mu = 0.2, n_traj = 10)
plot_traj(x, "NIG process with alpha = 0.5")

# 6. All the simulated data in a single data frame, with the increments
head(traj_df(x))
head(traj_df(x, increments = TRUE))
```

```
# 7. Histogram of the increments, with the density of the increments in red.
#    Constant step h = 5: the increments are i.i.d.
#    with distribution NIG(alpha, beta, delta h, mu h)
t7 <- inspection_times("regular", n = 20, tmax = 100)$time
x <- sim_nig(t7, alpha = 2, beta = 0.5, delta = 1, mu = 0.2, n_traj = 100)
dx <- traj_df(x, increments = TRUE)$dX
dx <- dx[!is.na(dx)]
dnig <- function(y, alpha = 2, beta = 0.5, delta = 1 * 5, mu = 0.2 * 5) {
  gamma <- sqrt(alpha^2 - beta^2)
  q <- sqrt(delta^2 + (y - mu)^2)
  alpha * delta / (pi * q) * besselK(alpha * q, 1, expon.scaled = TRUE) *
    exp(delta * gamma + beta * (y - mu) - alpha * q)
}
ggplot2::ggplot(data.frame(dx), ggplot2::aes(dx)) +
  ggplot2::geom_histogram(ggplot2::aes(y = ggplot2::after_stat(density)),
                          bins = 30, fill = "grey80", colour = "grey40") +
  ggplot2::stat_function(fun = dnig, colour = "red") +
  ggplot2::labs(x = "dX", y = "Density", title = "Increments of the NIG process")
```

`sim_vg` *Simulation of the variance-gamma process*

Description

Simulates exactly, at given inspection times, one or several trajectories of a variance-gamma (VG) process, a Levy process.

Usage

```
sim_vg(
  t,
  theta,
  sigma,
  nu,
  increments = "subordination",
  method = "RWA",
  type = NULL,
  n_traj = 1
)
```

Arguments

`t`	Inspection times. The times of each trajectory must start at 0 and be strictly increasing. Three forms are accepted: • a numeric vector: all the trajectories share these times, and their number is `n_traj`. The column `time` of the result of `inspection_times()` can be used; • a list of vectors, one per trajectory, for instance `list(c(0, 1, 3), c(0, 2, 4, 6))`: each trajectory has its own times (`n_traj` must then stay 1); • the data frame returned by `inspection_times()`, which gives one plan per trajectory (`n_traj` must then stay 1).

`theta` Drift of the subordinated Brownian motion, real: mean degradation per unit of time.

`sigma` Diffusion coefficient of the subordinated Brownian motion, $\sigma > 0$ (standard deviation, not variance).

`nu` Variance rate of the gamma process G, $\nu > 0$: the larger ν, the larger the jumps; as $\nu \to 0$ the process becomes a Wiener process.

`increments` Algorithm that draws each increment:

- `"subordination"` (default);
- `"difference"`: difference of two gamma processes.

Both are exact and give the same distribution; see the section Exact simulation.

`method` Simulation method, described in the section Simulation methods of DegradSim-package: `"RWA"` (random walk approximation, default) or `"Bridge"` (bridge sampling). The two give the same distribution, but different trajectories with the same seed. `"Series"` (truncated series representation) is not currently available and gives an error.

`type` Used only with `method = "Series"`, to choose the series representation; `NULL` (default).

`n_traj` Number of trajectories to simulate when `t` is a vector (default 1), a positive integer. It must stay 1 when `t` is a plan or a list, which already gives the trajectories: to simulate several trajectories on the times of a plan, give `plan$time`.

Value

An object of class `DegradSim`: a matrix (column `t`, then one column of levels per trajectory) when all the trajectories have the same inspection times, otherwise a list with one data frame (`t`, `X`) per trajectory. The model, the parameters and the time-scaling function, if any, are stored as attributes. The use of the result (a trajectory, its times and levels, `traj_df()`, `plot_traj()`) is described in the section Getting started of DegradSim-package.

Model

Let θ be real, $\sigma > 0$ and $\nu > 0$. Let G be a gamma process with $G_t - G_s \sim \Gamma((t-s)/\nu, \text{rate } 1/\nu)$, so that $E[G_t] = t$ and $Var(G_t) = \nu t$, and B an independent standard Brownian motion. The VG process is the Brownian motion with drift evaluated at the random time G_t given by the gamma process G:

$$X_t = \theta G_t + \sigma B_{G_t}, \qquad t \geq 0.$$

It is a Levy process: $X_0 = 0$, its increments are independent and stationary, $E[X_t] = \theta t$ and $Var(X_t) = (\sigma^2 + \theta^2\nu)\, t$. The trajectories are not monotone, purely made of jumps, of finite variation. As $\nu \to 0$, the VG process tends to the Wiener process $\theta t + \sigma B_t$.

Exact simulation

Random walk approximation, described in DegradSim-package: each increment, over a time $\Delta t_i = t_i - t_{i-1}$, is drawn by one of two exact algorithms (argument `increments`), which give the same distribution, then the increments are cumulated.

Algorithm 1, subordination (`increments = "subordination"`, default). Conditionally on the increment ΔG of the gamma process, $X_t - X_s \sim \mathcal{N}(\theta\Delta G, \sigma^2\Delta G)$:

1. draw independently $\Delta G_i \sim \Gamma(\Delta t_i/\nu)$ with scale ν;
2. compute $\Delta X_i = \theta\Delta G_i + \sigma\sqrt{\Delta G_i}\, Z_i$, with Z_i independent $\mathcal{N}(0,1)$.

This algorithm follows the definition of the process and also gives the gamma process G.

Algorithm 2, difference of two gamma processes (`increments = "difference"`). Let

$$\mu_{\pm} = \frac{1}{2}\sqrt{\theta^2 + \frac{2\sigma^2}{\nu}} \pm \frac{\theta}{2} > 0,$$

and G_+, G_- two independent gamma processes with shapes t/ν and scales $\nu\mu_{\pm}$. Then X has the same distribution as $G_+ - G_-$: since $\mu_+ - \mu_- = \theta$ and $\nu^2\mu_+\mu_- = \sigma^2\nu/2$, the characteristic function of $\Delta G_+ - \Delta G_-$ over a time Δt is

$$(1 - iu\nu\mu_+)^{-\Delta t/\nu}(1 + iu\nu\mu_-)^{-\Delta t/\nu} = \left(1 - iu\theta\nu + \sigma^2\nu u^2/2\right)^{-\Delta t/\nu},$$

which is the characteristic function of the VG increment (Madan et al., 1998).

1. Draw independently $\Delta G_i^+ \sim \Gamma(\Delta t_i/\nu)$ with scale $\nu\mu_+$ and $\Delta G_i^- \sim \Gamma(\Delta t_i/\nu)$ with scale $\nu\mu_-$;
2. compute $\Delta X_i = \Delta G_i^+ - \Delta G_i^-$.

This algorithm only uses gamma draws.

With `method = "Bridge"`, the values are drawn by `levyBridge::vg_bridge()`: see the section Simulation methods of DegradSim-package. The argument `increments` then only concerns the draw of the value at the last inspection time.

References

Madan, D. B., Carr, P. P. and Chang, E. C. (1998). The variance gamma process and option pricing. *European Finance Review*, 2(1), 79-105. doi:10.1023/A:1009703431535

See Also

`plot_traj()`, `traj_df()`, `inspection_times()`, `plot_plan()`

Examples

```
# 1. One trajectory with 20 regular inspections up to 100 (step 5)
t <- inspection_times("regular", n = 20, tmax = 100)$time
set.seed(1)
x <- sim_vg(t, theta = 0.5, sigma = 1.2, nu = 0.8)
x
plot_traj(x)

# 1b. One trajectory with 20 inspections up to 100 whose steps increase
#     (inspections less and less frequent)
t_inc <- inspection_times("increasing", n = 20, tmax = 100)$time
round(diff(t_inc), 2)
x <- sim_vg(t_inc, theta = 0.5, sigma = 1.2, nu = 0.8)
plot_traj(x)

# 2. Ten trajectories, simulated by subordination (default)
x <- sim_vg(t, theta = 0.5, sigma = 1.2, nu = 0.8, n_traj = 10)
plot_traj(x)

# Same times for all the trajectories: a matrix with the column t,
# then one column per trajectory (X1, X2, ...)
head(x[, 1:4])
```

```
# 3. The same model simulated as a difference of two gamma processes
y <- sim_vg(t, theta = 0.5, sigma = 1.2, nu = 0.8, increments = "difference", n_traj = 10)
plot_traj(y, "Variance-gamma process, difference of two gamma processes")

# 4. Irregular inspections, more and more frequent over time
plan <- inspection_times("decreasing", n = 20, n_traj = 5, tmax = 100)
x <- sim_vg(plan, theta = 0.5, sigma = 1.2, nu = 0.8)
plot_traj(x)

# 5. Different inspection times for each trajectory: a list of data frames
x <- sim_vg(list(c(0, 1, 3, 6), c(0, 2, 4, 6, 8, 10)), theta = 0.5, sigma = 1.2, nu = 0.8)
x[[2]]
x[[2]]$X

# 6. Small nu: the trajectories look like those of a Wiener process
x <- sim_vg(t, theta = 0.5, sigma = 1.2, nu = 0.01, n_traj = 10)
plot_traj(x, "Variance-gamma process with nu = 0.01")

# 7. All the simulated data in a single data frame, with the increments
head(traj_df(x))
head(traj_df(x, increments = TRUE))

# 8. Histogram of the increments, with the density of the increments in red.
#    Constant step h = 5: the increments are i.i.d.
#    with the VG density of Madan et al. (1998), written with besselK()
t8 <- inspection_times("regular", n = 20, tmax = 100)$time
x <- sim_vg(t8, theta = 0.5, sigma = 1.2, nu = 0.8, n_traj = 100)
dx <- traj_df(x, increments = TRUE)$dX
dx <- dx[!is.na(dx)]
dvg <- function(y, h = 5, theta = 0.5, sigma = 1.2, nu = 0.8) {
  r <- sqrt(theta^2 + 2 * sigma^2 / nu)
  z <- abs(y) * r / sigma^2
  2 / (nu^(h / nu) * sqrt(2 * pi) * sigma * gamma(h / nu)) * (abs(y) / r)^(h / nu - 0.5) *
    besselK(z, h / nu - 0.5, expon.scaled = TRUE) * exp(theta * y / sigma^2 - z)
}
ggplot2::ggplot(data.frame(dx), ggplot2::aes(dx)) +
  ggplot2::geom_histogram(ggplot2::aes(y = ggplot2::after_stat(density)),
                          bins = 30, fill = "grey80", colour = "grey40") +
  ggplot2::stat_function(fun = dvg, colour = "red") +
 ggplot2::labs(x = "dX", y = "Density", title = "Increments of the variance-gamma process")

# 9. Bridge sampling: one trajectory. The printed result gives the method
x <- sim_vg(t, theta = 0.5, sigma = 1.2, nu = 0.8, method = "Bridge")
x
plot_traj(x)

# 10. Bridge sampling: ten trajectories with the same inspection times
x <- sim_vg(t, theta = 0.5, sigma = 1.2, nu = 0.8, n_traj = 10, method = "Bridge")
plot_traj(x)
head(x[, 1:4])

# 11. Bridge sampling: irregular inspections, the same as above
x <- sim_vg(plan, theta = 0.5, sigma = 1.2, nu = 0.8, method = "Bridge")
plot_traj(x)

# 12. Bridge sampling: all the simulated data in a single data frame, with the increments
```

```
head(traj_df(x, increments = TRUE))

# 13. Bridge sampling: different inspection times for each trajectory
x <- sim_vg(list(c(0, 1, 3, 6), c(0, 2, 4, 6, 8, 10)), theta = 0.5, sigma = 1.2, nu = 0.8,
            method = "Bridge")
x[[2]]

# 14. Bridge sampling: last value drawn as a difference of two gamma processes
y <- sim_vg(t, theta = 0.5, sigma = 1.2, nu = 0.8, increments = "difference", n_traj = 10,
            method = "Bridge")
plot_traj(y, "Variance-gamma process, bridge")

# 15. Bridge sampling: histogram of the increments, with the same density as above
x <- sim_vg(t8, theta = 0.5, sigma = 1.2, nu = 0.8, n_traj = 100, method = "Bridge")
dx <- traj_df(x, increments = TRUE)$dX
dx <- dx[!is.na(dx)]
dvg <- function(y, h = 5, theta = 0.5, sigma = 1.2, nu = 0.8) {
  r <- sqrt(theta^2 + 2 * sigma^2 / nu)
  z <- abs(y) * r / sigma^2
  2 / (nu^(h / nu) * sqrt(2 * pi) * sigma * gamma(h / nu)) * (abs(y) / r)^(h / nu - 0.5) *
    besselK(z, h / nu - 0.5, expon.scaled = TRUE) * exp(theta * y / sigma^2 - z)
}
ggplot2::ggplot(data.frame(dx), ggplot2::aes(dx)) +
  ggplot2::geom_histogram(ggplot2::aes(y = ggplot2::after_stat(density)),
                          bins = 30, fill = "grey80", colour = "grey40") +
  ggplot2::stat_function(fun = dvg, colour = "red") +
  ggplot2::labs(x = "dX", y = "Density", title = "Variance-gamma process, bridge")
```

`sim_wiener` *Simulation of the Wiener process*

Description

Simulates exactly, at given inspection times, one or several trajectories of a Wiener process with drift (a Levy process) or of a time-scaled Wiener process, obtained with a time-scaling function A.

Usage

```
sim_wiener(t, mu, sigma, A = NULL, method = "RWA", type = NULL, n_traj = 1)
```

Arguments

`t`	Inspection times. The times of each trajectory must start at 0 and be strictly increasing. Three forms are accepted: • a numeric vector: all the trajectories share these times, and their number is `n_traj`. The column `time` of the result of `inspection_times()` can be used; • a list of vectors, one per trajectory, for instance `list(c(0, 1, 3), c(0, 2, 4, 6))`: each trajectory has its own times (`n_traj` must then stay 1); • the data frame returned by `inspection_times()`, which gives one plan per trajectory (`n_traj` must then stay 1).
`mu`	Drift of the Wiener process, real: mean degradation per unit of time. Not given when `A` is given (the mean of X_t is then $A(t)$).

`sigma`	Volatility $\sigma > 0$. The increment $X_t - X_s$ has variance $\sigma^2(t-s)$ for the Wiener process, and $\sigma^2(A(t) - A(s))$ for the time-scaled Wiener process.
`A`	`NULL` (default) for the gamma process, or the time-scaling function A of the time-scaled process (defined in DegradSim-package): a vectorized R function of `t`, for instance `function(t) 2 * t^1.5`.
`method`	Simulation method, described in the section Simulation methods of DegradSim-package: `"RWA"` (random walk approximation, default) or `"Bridge"` (bridge sampling). The two give the same distribution, but different trajectories with the same seed. `"Series"` (truncated series representation) is not currently available and gives an error.
`type`	Used only with `method = "Series"`, to choose the series representation; `NULL` (default).
`n_traj`	Number of trajectories to simulate when `t` is a vector (default 1), a positive integer. It must stay 1 when `t` is a plan or a list, which already gives the trajectories: to simulate several trajectories on the times of a plan, give `plan$time`.

Value

An object of class `DegradSim`: a matrix (column `t`, then one column of levels per trajectory) when all the trajectories have the same inspection times, otherwise a list with one data frame (`t`, `X`) per trajectory. The model, the parameters and the time-scaling function, if any, are stored as attributes. The use of the result (a trajectory, its times and levels, `traj_df()`, `plot_traj()`) is described in the section Getting started of DegradSim-package.

Model

B denotes a standard Brownian motion and $\mathcal{N}(m, v)$ the normal distribution with mean m and variance v.

Wiener process. Let μ be real and $\sigma > 0$. The process is

$$X_t = \mu t + \sigma B_t, \qquad t \geq 0,$$

where μ is the drift parameter (expected to be positive for a degradation) and σ^2 the variance (or volatility) parameter. It has independent increments $X_t - X_s \sim \mathcal{N}(\mu(t-s), \sigma^2(t-s))$, $E[X_t] = \mu t$ and $Var(X_t) = \sigma^2 t$. It is a Levy process with continuous, non-monotone trajectories.

Time-scaled Wiener process (`A` given). With the time-scaling function A defined in DegradSim-package,

$$X_t = A(t) + \sigma B_{A(t)}, \qquad t \geq 0,$$

that is $X_t = Y_{A(t)}$ with $Y_u = u + \sigma B_u$: the drift is set to $\mu = 1$ (identifiability, see DegradSim-package), as in Cha and Mercier (2019, Example 1). Its increments are independent, $X_t - X_s \sim \mathcal{N}(A(t) - A(s), \sigma^2(A(t) - A(s)))$, $E[X_t] = A(t)$ and $Var(X_t) = \sigma^2 A(t)$.

Exact simulation

Random walk approximation, described in DegradSim-package (Kahle et al., 2016, algorithm 1.7): the increments $\Delta X_i = \mu \Delta t_i + \sigma\sqrt{\Delta t_i}\, Z_i$ or $\Delta X_i = \Delta A_i + \sigma\sqrt{\Delta A_i}\, Z_i$, with Z_i independent $\mathcal{N}(0,1)$ drawn with `stats::rnorm()`, are cumulated.

With `method = "Bridge"`, the values are drawn by `levyBridge::wiener_bridge()`: see the section Simulation methods of DegradSim-package.

References

Kahle, W., Mercier, S. and Paroissin, C. (2016). *Degradation Processes in Reliability*. ISTE and Wiley, London. ISBN 978-1-84821-888-8.

Cha, J. H. and Mercier, S. (2019). Transformed Lévy processes as state-dependent wear models. *Advances in Applied Probability*, 51(2), 468-486. doi:10.1017/apr.2019.21

Whitmore, G. A. (1995). Estimating degradation by a Wiener diffusion process subject to measurement error. *Lifetime Data Analysis*, 1, 307-319.

See Also

- `sim_gamma()`, `sim_ig()`: the other time-scaled processes;
- `plot_traj()`, `traj_df()`, `inspection_times()`, `plot_plan()`.

Examples

```
# 1. One trajectory with 20 regular inspections up to 100 (step 5)
t <- inspection_times("regular", n = 20, tmax = 100)$time
set.seed(1)
x <- sim_wiener(t, mu = 1.5, sigma = 2)
x
plot_traj(x)

# 1b. One trajectory with 20 inspections up to 100 whose steps increase
#     (inspections less and less frequent)
t_inc <- inspection_times("increasing", n = 20, tmax = 100)$time
round(diff(t_inc), 2)
x <- sim_wiener(t_inc, mu = 1.5, sigma = 2)
plot_traj(x)

# 2. Ten trajectories with the same inspection times
x <- sim_wiener(t, mu = 1.5, sigma = 2, n_traj = 10)
plot_traj(x)

# Same times for all the trajectories: a matrix with the column t,
# then one column per trajectory (X1, X2, ...)
head(x[, 1:4])

# 3. Irregular inspections, frequent at the beginning and at the end
plan <- inspection_times("inc-dec", n = 20, n_traj = 5, tmax = 100)
x <- sim_wiener(plan, mu = 1.5, sigma = 2)
plot_traj(x)

# 4. Different inspection times for each trajectory: a list of data frames
x <- sim_wiener(list(c(0, 1, 3, 6), c(0, 2, 4, 6, 8, 10)), mu = 1.5, sigma = 2)
x[[2]]
x[[2]]$X

# 5. Time-scaled Wiener process (model of Cha and Mercier, 2019, Example 1):
#    X_t = A(t) + sigma B_{A(t)}, here with A(t) = 0.5 t^1.5
x <- sim_wiener(t, sigma = 2, A = function(t) 0.5 * t^1.5, n_traj = 10)
x
plot_traj(x, "Time-scaled Wiener process")

# 6. With A(t) = mu t, the time-scaled Wiener process with sigma is the
#    Wiener process with drift mu and volatility sigma sqrt(mu)
```

```
set.seed(2)
y1 <- sim_wiener(t, sigma = 2, A = function(t) 1.5 * t)
set.seed(2)
y2 <- sim_wiener(t, mu = 1.5, sigma = 2 * sqrt(0.5))
all.equal(unclass(y1)[, 2], unclass(y2)[, 2])

# 7. All the simulated data in a single data frame, with the increments
head(traj_df(x))
head(traj_df(x, increments = TRUE))

# 8. Histogram of the increments, with their density in red. Constant
#    step h = 5: the increments are i.i.d. N(mu h, sigma^2 h)
t8 <- inspection_times("regular", n = 20, tmax = 100)$time
x <- sim_wiener(t8, mu = 1.5, sigma = 2, n_traj = 100)
dx <- traj_df(x, increments = TRUE)$dX
dx <- dx[!is.na(dx)]
ggplot2::ggplot(data.frame(dx), ggplot2::aes(dx)) +
  ggplot2::geom_histogram(ggplot2::aes(y = ggplot2::after_stat(density)),
                          bins = 30, fill = "grey80", colour = "grey40") +
  ggplot2::stat_function(fun = function(y) dnorm(y, 1.5 * 5, 2 * sqrt(5)),
                         colour = "red") +
  ggplot2::labs(x = "dX", y = "Density", title = "Increments of the Wiener process")

# 9. Bridge sampling: one trajectory. The printed result gives the method
x <- sim_wiener(t, mu = 1.5, sigma = 2, method = "Bridge")
x
plot_traj(x)

# 10. Bridge sampling: ten trajectories with the same inspection times
x <- sim_wiener(t, mu = 1.5, sigma = 2, n_traj = 10, method = "Bridge")
plot_traj(x)
head(x[, 1:4])

# 11. Bridge sampling: irregular inspections, the same as above
x <- sim_wiener(plan, mu = 1.5, sigma = 2, method = "Bridge")
plot_traj(x)

# 12. Bridge sampling: all the simulated data in a single data frame, with the increments
head(traj_df(x, increments = TRUE))

# 13. Bridge sampling: different inspection times for each trajectory
x <- sim_wiener(list(c(0, 1, 3, 6), c(0, 2, 4, 6, 8, 10)), mu = 1.5, sigma = 2,
                method = "Bridge")
x[[2]]

# 14. Bridge sampling: histogram of the increments, with the same density as above
x <- sim_wiener(t8, mu = 1.5, sigma = 2, n_traj = 100, method = "Bridge")
dx <- traj_df(x, increments = TRUE)$dX
dx <- dx[!is.na(dx)]
ggplot2::ggplot(data.frame(dx), ggplot2::aes(dx)) +
  ggplot2::geom_histogram(ggplot2::aes(y = ggplot2::after_stat(density)),
                          bins = 30, fill = "grey80", colour = "grey40") +
  ggplot2::stat_function(fun = function(y) dnorm(y, 1.5 * 5, 2 * sqrt(5)),
                         colour = "red") +
 ggplot2::labs(x = "dX", y = "Density", title = "Increments of the Wiener process, bridge")

# 15. Bridge sampling: time-scaled process with A(t) = 0.2 t^1.5
```

```
x <- sim_wiener(t8, sigma = 2, A = function(t) 0.2 * t^1.5, method = "Bridge", n_traj = 5)
plot_traj(x)
```

`traj_df` *Simulated data as a data frame*

Description

Gathers the trajectories simulated by a `sim_` function into a single data frame, one row per inspection, ready to be saved, summarized or used by other functions. Optionally adds the time steps and the increments between two consecutive inspections.

Usage

```
traj_df(x, increments = FALSE)
```

Arguments

`x`	Result of a simulation function of the package, for instance `sim_gamma()`.
`increments`	`FALSE` (default) for the levels only; `TRUE` to add the columns `dt` and `dX`.

Value

A data frame with one row per inspection and the columns:

- `id`: number of the trajectory (1, 2, ...);
- `t`: inspection time;
- `X`: simulated degradation level at time `t`;
- with `increments = TRUE`, also `dt`, the time elapsed since the previous inspection, and `dX`, the increment of the level since the previous inspection (`NA` at the first inspection, `t = 0`, of each trajectory).

To work on a single trajectory, use `subset(traj_df(x), id == k)`.

See Also

`plot_traj()`

Examples

```
set.seed(1)
x <- sim_gamma(inspection_times("regular", n = 20, tmax = 100)$time, a = 2, b = 3, n_traj = 3)
d <- traj_df(x)
head(d)

# Degradation level of each trajectory at the last inspection
d[d$t == 100, ]

# Time steps and increments between two inspections
head(traj_df(x, increments = TRUE))

# Trajectories with different inspection times
```

```
y <- sim_gamma(list(c(0, 1, 3, 6), c(0, 2, 4, 6, 8)), a = 2, b = 3)
traj_df(y, increments = TRUE)

# Save the simulated data in a csv file
f <- tempfile(fileext = ".csv")
write.csv(d, f, row.names = FALSE)
```

Index

∗ **package**
DegradSim-package, 2

DegradSim *(*DegradSim-package*)*, 2
DegradSim-package, 2, *13, 14*, *16, 17*, *19, 20*, *23, 24*, *27*, *30, 31*, *34*

inspection_times, 8
inspection_times(), *2*, *4, 5*, *11*, *13–17*, *19*, *21*, *23, 24*, *26*, *28, 29*, *31*, *33*, *35*

levyBridge::gamma_bridge(), *5*, *17*
levyBridge::ig_bridge(), *5*, *20*
levyBridge::meixner_bridge(), *5*, *24*
levyBridge::vg_bridge(), *5*, *31*
levyBridge::wiener_bridge(), *5*, *34*

plot_plan, 10
plot_plan(), *5*, *8, 9*, *14*, *17*, *21*, *24*, *28*, *31*, *35*
plot_traj, 11
plot_traj(), *5*, *11*, *13, 14*, *16, 17*, *20, 21*, *23, 24*, *27, 28*, *30, 31*, *34, 35*, *37*

sim_cpoisson, 12
sim_cpoisson(), *3*
sim_gamma, 15
sim_gamma(), *2–4*, *8, 9*, *12*, *14*, *21*, *35*, *37*
sim_ig, 19
sim_ig(), *3, 4*, *17*, *27*, *35*
sim_meixner, 23
sim_meixner(), *3*
sim_nig, 26
sim_nig(), *3*
sim_vg, 29
sim_vg(), *3*, *28*
sim_wiener, 33
sim_wiener(), *3, 4*, *17*, *21*
stats::rgamma(), *17*
stats::rnorm(), *34*
stats::rpois(), *14*

traj_df, 37
traj_df(), *5*, *12–14*, *16, 17*, *20, 21*, *23, 24*, *27, 28*, *30, 31*, *34, 35*